

Doc. No.:	P0941R0
Date:	2018-02-09
Reply to:	Clark Nelson
Audience:	SG10, EWG, LEWG

Integrating feature-test macros into the C++ WD

[-1] Preface

- [1] This revision of this paper is a hastily-prepared starting point, describing proposed directions for incorporating feature-test macros into the working draft of the C++ standard.
- [2] Obviously, its life started by incorporating the text of SD-6. I have deleted all the rationale and examples for individual features, and written this new preface. Other than that, virtually all the changes are added **editorial notes** suggesting what should be done with the text that follows; this is mainly to make life easier for both the reader and the author.

[0] Contents

1. [Introduction](#)
2. [Explanation and rationale for the approach](#)
 1. [Problem statement](#)
 2. [Status quo](#)
 3. [Characteristics of the proposed solution](#)
3. [Recommendations](#)
 1. [Introduction](#)
 2. [Testing for the presence of a header: `\_has\_include`](#)
 3. [Testing for the presence of an attribute: `\_has\_cpp\_attribute`](#)
 4. [C++17 features](#)
 5. [C++14 features](#)
 6. [C++11 features](#)
 7. [C++98 features](#)
 8. [Features published and later removed](#)
4. [Annex: Recommendations from Technical Specifications](#)
5. [Annex: Model wording for a Technical Specification](#)
6. [Revision history](#)

[1] Introduction

- [1] **There would be no point in adding this, or anything like it, to the WD.**
- [2] At the September 2013 (Chicago) meeting of WG21, there was a five-way poll of all of the C++ experts in attendance – approximately 80 – concerning their support for the approach described herein for feature-testing in C++. The results of the poll:

Strongly favor	Favor	Neutral	Oppose	Strongly oppose
lots	lots	1	0	0

- [3] General support for these recommendations was reaffirmed at the 2015 October (Kona) meeting, as indicated by the following straw poll:
- [4] Should WG21 encourage, but not require, feature-test macros for proposals?

Strongly favor	Favor	Neutral	Against	Strongly against
50	13	7	3	2

- [5] This document was subsequently designated WG21's SD-6 (sixth standing document), which will continue to be maintained by SG10.

[2] Explanation and rationale for the approach

- [1] **Probably some parts of this section should become a new informative annex.**

[2.1] Problem statement

- [1] **Some of this may make sense in the annex, or it may be better to rewrite it.**
- [2] The pace of innovation in the standardization of C++ makes long-term stability of implementations unlikely. Features are added to the language because programmers want to use those features. Features are added to (the working draft of) the standard as the features become well-specified. In many cases a feature is added to an implementation well before or well after the standard officially introducing it is approved.
- [3] This process makes it difficult for programmers who want to use a feature to know whether it is available in any given implementation. Implementations rarely leap from one formal revision of the standard directly to the next; the implementation process generally proceeds by smaller steps. As a result, testing for a specific revision of the standard (e.g. by examining the value of the `__cplusplus` macro) often gives the wrong answer. Implementers generally don't want to appear

to be claiming full conformance to a standard revision until all of its features are implemented. That leaves programmers with no portable way to determine which features are actually available to them.

[4] It is often possible for a program to determine, in a manner specific to a single implementation, what features are supported by that implementation; but the means are often poorly documented and ad hoc, and sometimes complex – especially when the availability of a feature is controlled by an invocation option. To make this determination for a variety of implementations in a single source base is complex and error-prone.

[2.2] Status quo

[1] This section should probably not be added to the WD.

[2] Here is some code that attempts to determine whether rvalue references are available in the implementation in use:

```
#ifndef __USE_RVALUE_REFERENCES
  #if ( __GNUC__ > 4 || __GNUC__ == 4 && __GNUC_MINOR__ >= 3 ) || \
      _MSC_VER >= 1600
    #if __EDG_VERSION__ > 0
      #define __USE_RVALUE_REFERENCES ( __EDG_VERSION__ >= 410 )
    #else
      #define __USE_RVALUE_REFERENCES 1
    #endif
  #elif __clang__
    #define __USE_RVALUE_REFERENCES __has_feature(cxx_rvalue_references)
  #else
    #define __USE_RVALUE_REFERENCES 0
  #endif
#endif
```

[3] First, the GNU and Microsoft version numbers are checked to see if they are high enough. But then a check is made of the EDG version number, since that front end also has compatibility modes for both those compilers, and defines macros indicating (claimed) compatibility with them. If the feature wasn't implemented in the indicated EDG version, it is assumed that the feature is not available – even though it is possible for a customer of EDG to implement a feature before EDG does.

[4] Fortunately Clang has ways to test specifically for the presence of specific features. But unfortunately, the function-call-like syntax used for such tests won't work with a standard preprocessor, so this fine new feature winds up adding its own flavor of complexity to the mix.

[5] Also note that this code is only the beginning of a real-world solution. A complete solution would need to take into account more compilers, and also command-line option settings specific to various compilers.

[2.3] Characteristics of the proposed solution

[1] This section should probably be incorporated into the informative annex much as it is, although it's easy to imagine that it could be improved by tweaks and additions.

[2] To preserve implementers' freedom to add features in the order that makes the most sense for themselves and their customers, implementers should indicate the availability of each separate feature by adding a definition of a macro with the name corresponding to that feature.

[3] **Important note:** By recommending the use of these macros, WG21 is **not** making any feature optional; the absence of a definition for the relevant feature-test macro does not make an implementation that lacks a feature conform to a standard that requires the feature. However, if implementers and programmers follow these recommendations, portability of code between real-world implementations should be improved.

[4] To a first approximation, a feature is identified by the WG21 paper in which it is specified, and by which it is introduced into the working draft of the standard. Not every paper introduces a new feature worth a feature-test macro, but every paper that is not just a collection of issue resolutions is considered a candidate; exceptions are explicitly justified.

[5] For C++14, the feature-test macro name generally consists of some combination of words from the title of the paper. In the future, it is hoped that every paper will include its own recommendations concerning feature-test macro names.

[6] The value specified for a feature-test macro is based on the year and month in which the feature is voted into the working draft. In a case where a feature is subsequently changed in a significant way, but arguably remains the same feature, the value of the macro is changed to indicate the “revision level” of the specification of the feature. However, in most cases it is expected that the presence of a feature can be determined by the presence of any non-zero macro value; for example:

```
template<typename T>
struct use_empty_base_opt :
    std::integral_constant<bool,
        std::is_empty<T>::value
    >
{
    #if __cpp_lib_is_final
        && !std::is_final<T>::value
    #endif
};
```

[7] To avoid the user's namespace, names of macros for language features are prefixed by “__cpp_”; for library features, by “__cpp_lib_”. A library feature that doesn't introduce a new header is expected to be defined by the header(s) that implement the feature.

[3] Recommendations

[1] Most of this section should become a new normative annex.

[3.1] Introduction

[1] This section would have to be modified to describe requirements (“shall”) instead of recommendations (“should”).

[2] For the sake of improved portability between partial implementations of various C++ standards, WG21 (the ISO technical committee for the C++ programming language) recommends that implementers and programmers follow the guidelines in this document concerning feature-test macros.

[3] Implementers who provide a new standard feature should define a macro with the recommended name and value, in the same circumstances under which the feature is available (for example, taking into account relevant command-line options), to indicate the presence of support for that feature.

[4] Programmers who wish to determine whether a feature is available in an implementation should base that determination on the state of the macro with the recommended name. (The absence of a tested feature may result in a program with decreased functionality, or the relevant functionality may be provided in a different way. A program that strictly depends on support for a feature can just try to use the feature unconditionally; presumably, on an implementation lacking necessary support, translation will fail. Therefore, if the most useful purpose for a feature-test macro would be to control the inclusion of a `#error` directive if the feature is unavailable, that is considered inadequate justification for the macro. Note that the usefulness of a test macro for a feature is completely independent of the usefulness of the feature itself.)

[3.2] Testing for the presence of a header: `__has_include`

[1] This section was already incorporated into C++17, so it should be ignored.

[2] It is impossible for a C++ program to directly, reliably and portably determine whether or not a library header is available for inclusion. Conditionally including a header requires the use of a configuration macro, whose setting can be determined by a configuration-test process at build time (reliable, but less portable), or by some other means (often not reliable or portable).

[3] To solve this general problem, WG21 recommends that implementers provide, and programmers use, the `__has_include` feature.

[3.2.1] Syntax

h-preprocessing-token:

any *preprocessing-token* other than `>`

h-pp-tokens:

h-preprocessing-token

h-pp-tokens *h-preprocessing-token*

has-include-expression:

`__has_include ( header-name )`

`__has_include ( string-literal )`

`__has_include ( < h-pp-tokens > )`

[3.2.2] Semantics

[1] In the first form of the *has-include-expression*, the parenthesized *header-name* token is not subject to macro expansion. The second and third forms are considered only if the first form does not match, and the preprocessing tokens are processed just as in normal text.

[2] A *has-include-expression* shall appear only in the controlling constant expression of a `#if` or `#elif` directive ([cpp.cond] 16.1). Prior to the evaluation of such an expression, the source file identified by the parenthesized preprocessing token sequence in each contained *has-include-expression* is searched for as if that preprocessing token sequence were the *pp-tokens* in a `#include` directive, except that no further macro expansion is performed. If such a directive would not satisfy the syntactic requirements of a `#include` directive, the program is ill-formed. The *has-include-expression* is replaced by the *pp-number* 1 if the search for the source file succeeds, and by the *pp-number* 0 if the search fails.

[3] The `#ifdef` and `#ifndef` directives, and the `defined` conditional inclusion operator, shall treat `__has_include` as if it were the name of a defined macro. The identifier `__has_include` shall not appear in any context not mentioned in this section.

[3.2.3] Example

[1] This demonstrates a way to use a library optional facility only if it is available.

```
#ifdef __has_include
# if __has_include(<optional>)
#   include <optional>
#   define have_optional 1
# elif __has_include(<experimental/optional>)
#   include <experimental/optional>
#   define have_optional 1
#   define experimental_optional
# else
#   define have_optional 0
# endif
#endif
```

[3.3] Testing for the presence of an attribute: `__has_cpp_attribute`

[1] This section should be incorporated into the description of the preprocessor.

[2] A C++ program cannot directly, reliably, and portably determine whether or not a standard or vendor-specific attribute is available for use. Testing for attribute support generally requires complex macro logic, as illustrated above for language features in general.

[3] To solve this general problem, WG21 recommends that implementers provide, and programmers use, the `__has_cpp_attribute` feature.

[3.3.1] Syntax

has-attribute-expression:

```
__has_cpp_attribute ( attribute-token )
```

[3.3.2] Semantics

[1] A *has-attribute-expression* shall appear only in the controlling constant expression of a `#if` or `#elif` directive ([cpp.cond] 16.1). The *has-attribute-expression* is replaced by a non-zero pp-number if the implementation supports an attribute with the specified name, and by the pp-number 0 otherwise.

[2] For a standard attribute, the value of the `__has_cpp_attribute` macro is based on the year and month in which the attribute was voted into the working draft. In the case where the attribute is vendor-specific, the value is implementation-defined. However, in most cases it is expected that the availability of an attribute can be detected by any non-zero result.

[3] The `#ifdef` and `#ifndef` directives, and the `defined` conditional inclusion operator, shall treat `__has_cpp_attribute` as if it were the name of a defined macro. The identifier `__has_cpp_attribute` shall not appear in any context not mentioned in this section.

[3.3.3] Example

[1] This demonstrates a way to use the attribute `[[deprecated]]` only if it is available.

```
#ifndef __has_cpp_attribute
# define __has_cpp_attribute(x) 0
#endif
#if __has_cpp_attribute(deprecated)
# define ATTR_DEPRECATED(msg) [[deprecated(msg)]]
#else
# define ATTR_DEPRECATED(msg)
#endif
```

[3.4] C++17 features

[1] This paragraph probably isn't needed in the WD.

[2] The following table itemizes all the changes that were made to the working draft for C++17 as specified in a WG21 technical document. (Changes that were made as specified in a core or library issue are not generally included.)

[3] Whether the "Primary Section" column should appear, how the table should be sorted, and what links should be included in the WD, are all editorial questions that will need to be decided.

[4] The table is sorted by the section of the standard primarily affected. The "Doc. No." column links to the paper itself on the committee web site. The "Macro Name" column links to the relevant portion of the "Detailed explanation and rationale" annex of this document. When the recommendation is to change the value of a macro previously recommended to be defined, the "Value" column links to the table entry for the previous recommendation.

[5] The most important change here would be that any library macro be defined by including `<version>`, as well as the header specified in the table.

[6] For library features, the "Header" column identifies the header that is expected to define the macro, although the macro may also be predefined. For language features, the macro must be predefined.

Significant changes to C++17

Doc. No.	Title	Primary Section	Macro Name	Value	Header
P0296R2	Forward progress guarantees: Base definitions	4.7	none		
P0299R1	Forward progress guarantees for the Parallelism TS features	4.7, 28.4	none		
N4086	Removing trigraphs??!	5	none		
P0245R1	Hexadecimal floating literals for C++	5.13	__cpp_hex_float ex.	201603	<i>predefined</i>
N4267	Adding u8 character literals	5.13	none		
P0386R2	Inline Variables	6.2, 10.1	__cpp_inline_variables	201606	<i>predefined</i>
P0035R4	Dynamic memory allocation for over-aligned data	6.7, 8.3, 21.6	__cpp_aligned_new ex.	201606	<i>predefined</i>
P0135R1	Wording for guaranteed copy elision through simplified value categories	6.10	__cpp_guaranteed_copy_elision	201606	<i>predefined</i>
N4261	Proposed resolution for Core Issue 330: Qualification conversions and pointers to arrays of pointers	7.5, 8.2	none		
P0012R1	Make exception specifications be part of the type system	7.13, 18.4	__cpp_noexcept_function_type ex.	201510	<i>predefined</i>
P0145R3	Refining Expression Evaluation Order for Idiomatic C++	8	none		
N4295	Folding expressions	8.1, 17.6	__cpp_fold_expressions	201411	<i>predefined</i>
P0018R3	Lambda Capture of <code>*this</code> by Value as <code>[=,*this]</code>	8.1	__cpp_capture_star_this ex.	201603	<i>predefined</i>
P0170R1	Wording for <code>constexpr</code> Lambda	8.1	__cpp_constexpr	201603	<i>predefined</i>
P0002R1	Remove Deprecated operator++(bool)	8.3	none		
P0292R2	<code>constexpr</code> if: A slightly different syntax	9.4	__cpp_if_constexpr ex.	201606	<i>predefined</i>
P0305R1	Selection statements with initializer	9.4	none		

Doc. No.	Title	Primary Section	Macro Name	Value	Header
P0184R0	Generalizing the Range-Based For Loop	9.5	__cpp_range_based_for	201603	predefined
N3928	Extending static_assert	10	__cpp_static_assert	201411	predefined
N3922	New Rules for auto deduction from braced-init-list	10.1	none		
P0001R1	Remove Deprecated Use of the register Keyword	10.1	none		
P0091R3	Template argument deduction for class templates	10.1, 16.3, 17.9	__cpp_deduction_guides ex.	201606	predefined
P0512R0	Class Template Argument Deduction Assorted NB resolution and issues	16.3, 17.9	__cpp_deduction_guides ex.	201611	predefined
P0127R2	Declaring non-type template parameters with auto	10.1, 17.8	__cpp_nontype_template_parameter_auto ex.	201606	predefined
N4266	Attributes for namespaces and enumerators	10.2, 10.3	__cpp_namespace_attributes	201411	predefined
			__cpp_enumerator_attributes	201411	predefined
N4230	Nested namespace definition	10.3	none		
P0136R1	Rewording inheriting constructors (core issue 1941 et al)	10.3, 15.6	__cpp_inheriting_constructors ex.	201511	predefined
P0195R2	Pack expansions in using-declarations	10.3	__cpp_variadic_using	201611	predefined
P0188R1	Wording for [[fallthrough]] attribute	10.6	_has_cpp_attribute(fallthrough)	201603	predefined
P0189R1	Wording for [[nodiscard]] attribute	10.6	_has_cpp_attribute(nodiscard)	201603	predefined
P0212R1	Wording for [[maybe_unused]] attribute	10.6	_has_cpp_attribute(maybe_unused)	201603	predefined
P0028R4	Using attribute namespaces without repetition	10.6	none		
P0283R2	Standard and non-standard attributes	10.6	none		
P0217R3	Proposed wording for structured bindings	11.5	__cpp_structured_bindings ex.	201606	predefined
P0017R1	Extension to aggregate initialization	11.6	__cpp_aggregate_bases ex.	201603	predefined
P0138R2	Construction Rules for enum class Values	11.6	none		
P0398R0	Core issue 1518: Explicit default constructors and copy-list-initialization	11.6	none		
P0134R0	Introducing a name for brace-or-equal-initializers for non-static data members	12.2	none		
P0391R0	Introducing the term "templated entity"	17	none		
N4051	Allow typename in a template template parameter	17.1	none		
N4268	Allow constant evaluation for all non-type template arguments	17.3	__cpp_nontype_template_args	201411	predefined
P0522R0	Matching of template template-arguments excludes compatible templates	17.3	__cpp_template_template_args ex.	201611	predefined
P0036R0	Unary Folds and Empty Parameter Packs	17.6	__cpp_fold_expressions	201603	predefined
N4262	Wording for Forwarding References	17.9	none		
N4285	Cleanup for exception-specification and throw-expression	18	none		
P0003R5	Removing Deprecated Exception Specifications from C++17	18.4	none		
P0061R1	_has_include for C++17	19.1	_has_include	defined	predefined
P0063R3	C++17 should refer to C11 instead of C99	20.5	none		
P0180R2	Reserve a New Library Namespace Future Standardization	20.5	none		
P0175R1	Synopses for the C library	21 etc.	none		
P0298R3	A byte type definition	21.2	__cpp_lib_byte ex.	201603	<cstdint>
P0154R1	constexpr std::hardware_{constructive,destructive}_interference_size	21.6	__cpp_lib_hardware_interference_size ex.	201703	<new>
P0137R1	Core Issue 1776: Replacement of class objects containing reference members	21.6, 4.5	__cpp_lib_launder ex.	201606	<new>
N4259	Wording for std::uncaught_exceptions	21.8	__cpp_lib_uncaught_exceptions	201411	<exception>
N4190	Removing auto_ptr, random_shuffle(), And Old <functional> Stuff	23	none		
P0007R1	Constant View: A proposal for a std::as_const helper function template	23.2	__cpp_lib_as_const ex.	201510	<utility>
N4387	Improving pair and tuple	23.4, 23.5	none		
P0209R2	make_from_tuple: apply for construction	23.5	__cpp_lib_make_from_tuple ex.	201606	<tuple>

Doc. No.	Title	Primary Section	Macro Name	Value	Header
P0220R1	Adopt Library Fundamentals V1 TS Components for C++17	23.5	_cpp_lib_apply ex.	201603	<tuple>
		23.6	_has_include(<optional>)	1	<i>predefined</i>
			_cpp_lib_optional	201603	<optional>
		23.8	_has_include(<any>)	1	<i>predefined</i>
			_cpp_lib_any	201603	<any>
		23.12	_has_include(<memory_resource>)	1	<i>predefined</i>
		23.14	_cpp_lib_memory_resource	201603	<memory_resource>
			_cpp_lib_boyer_moore_searcher ex.	201603	<functional>
P0032R3	Homogeneous interface for variant, any and optional	23.6-8	_has_include(<string_view>)	1	<i>predefined</i>
			_cpp_lib_string_view	201603	<string_view>
			_cpp_lib_sample ex.	201603	<algorithm>
			_lib_cpp_any	201606	<any>
P0307R2	Making Optional Greater Equal Again	23.6	_cpp_lib_optional	201606	<optional>
P0088R3	Variant: a type-safe union for C++17	23.7	_has_include(<variant>)	1	<i>predefined</i>
			_cpp_lib_variant ex.	201606	<variant>
P0393R3	Making Variant Greater Equal	23.7	<i>none</i>		
LWG2296	std::addressof should be constexpr	23.10	_cpp_lib_addressof_constexpr	201603	<memory>
P0040R3	Extending memory management tools	23.10	_cpp_lib_raw_memory_algorithms	201606	<memory>
P0174R2	Deprecating Vestigial Library Parts in C++17	23.10, 27.3	<i>none</i>		
P0074R0	Making std::owner_less more flexible	23.11	_cpp_lib_transparent_operators	201510	<memory> <functional>
N4089	Safe conversions in unique_ptr<T[]>	23.11	<i>none</i>		
N4366	LWG 2228: Missing SFINAE rule in unique_ptr templated assignment	23.11	<i>none</i>		
P0033R1	Re-enabling shared_from_this	23.11	_cpp_lib_enable_shared_from_this	201603	<memory>
P0163R0	shared_ptr::weak_type	23.11	_cpp_lib_shared_ptr_weak_type	201606	<memory>
P0497R0	Fixes to shared_ptr support for arrays	23.11	_cpp_lib_shared_ptr_arrays ex.	201611	<memory>
P0337R0	Delete operator= for polymorphic_allocator	23.12	<i>none</i>		
N4169	A proposal to add invoke function template	23.14	_cpp_lib_invoke	201411	<functional>
N4277	TriviallyCopyable reference_wrapper	23.14	<i>none</i>		
P0005R4	Adopt not_fn from Library Fundamentals 2 for C++17	23.14	_cpp_lib_not_fn ex.	201603	<functional>
P0358R1	Fixes for not_fn	23.14	<i>none</i>		
P0253R1	Fixing a design mistake in the searchers interface in Library Fundamentals	23.14	<i>none</i>		
P0302R1	Removing Allocator Support in std::function	23.14	<i>none</i>		
N3911	TransformationTrait Alias void_t	23.15	_cpp_lib_void_t	201411	<type_traits>
N4389	Wording for bool_constant	23.15	_cpp_lib_bool_constant	201505	<type_traits>
P0006R0	Adopt Type Traits Variable Templates from Library Fundamentals TS for C++17	23.15	_cpp_lib_type_trait_variable_templates ex.	201510	<type_traits>
P0013R1	Logical Operator Type Traits	23.15	_cpp_lib_logical_traits	201510	<type_traits>
P0185R1	Adding [nothrow_]swappable traits	23.15	_cpp_lib_is_swappable	201603	<type_traits>
P0604R0	Resolving GB 55, US 84, US 85, US 86	23.15	_cpp_lib_is_invocable ex.	201703	<type_traits>
P0258R2	has_unique_object_representations - wording	23.15	_cpp_lib_has_unique_object_representations ex.	201606	<type_traits>
LWG2911	An is_aggregate type trait is needed	23.15	_cpp_lib_is_aggregate ex.	201703	<type_traits>
P0092R1	Polishing <chrono>	23.17	_cpp_lib_chrono ex.	201510	<chrono>
P0505R0	Wording for GB 50	23.17	_cpp_lib_chrono ex.	201611	<chrono>
P0024R2	The Parallelism TS Should be Standardized	23.19	_has_include(<execution>)	1	<i>predefined</i>
			_lib_cpp_execution	201603	<execution>
		28, 29	_cpp_lib_parallel_algorithm ex.	201603	<algorithm> <numeric>
P0336R1	Better Names for Parallel Execution Policies in C++17	23.19	<i>none</i>		
P0067R5	Elementary string conversions	23.20	_cpp_lib_to_chars ex.	201611	<utility>
P0254R2	Integrating std::string_view and std::string	24.3, 24.4	_cpp_lib_string_view	201606	<string> <string_view>

Doc. No.	Title	Primary Section	Macro Name	Value	Header
P0272R1	Give 'std::string' a non-const '.data()' member function	24.3	none		
P0618R0	Deprecating <codecvt>	25.4	none		
N4258	Cleaning-up noexcept in the Library	26.3- 26.5, 24.3	_cpp_lib_allocator_traits_is_always_equal	201411	<memory> <scoped_allocator> <string> <deque> <forward_list> <list> <vector> <map> <set> <unordered_map> <unordered_set>
N4510	Minimal incomplete type support for standard containers	26.3, 20.5	_cpp_lib_incomplete_container_elements ex.	201505	<forwardlist> <list> <vector>
P0084R2	Emplace Return Type	26.3	none		
N4279	Improved insertion interface for unique-key maps	26.4	_cpp_lib_map_try_emplace	201411	<map>
		26.5	_cpp_lib_unordered_map_try_emplace	201411	<unordered_map>
P0083R3	Splicing Maps and Sets	26.2	_cpp_lib_node_extract	201606	<map> <set> <unordered_map> <unordered_set>
N4284	Contiguous Iterators	27.2	none		
P0031R0	A Proposal to Add Constexpr Modifiers to reverse_iterator, move_iterator, array and Range Access	27.3	_cpp_lib_array_constexpr ex.	201603	<iterator> <array>
N4280	Non-member size() and more	27.8	_cpp_lib_nonmember_container_access	201411	<iterator> <array> <deque> <forward_list> <list> <map> <regex> <set> <string> <unordered_map> <unordered_set> <vector>
P0394R4	Hotel Parallelifornia: terminate() for Parallel Algorithms Exception Handling	28.4, 21.8	none		
P0025R0	An algorithm to "clamp" a value between a pair of boundary values	28.7	_cpp_lib_clamp ex.	201603	<algorithm>
P0346R1	A <random> Nomenclature Tweak	29.6	none		
P0295R0	Adopt Selected Library Fundamentals V2 Components for C++17	29.8	_cpp_lib_gcd_lcm ex.	201606	<numeric>
P0030R1	Proposal to Introduce a 3-Argument Overload to std::hypot	29.9	_cpp_lib_hypot ex.	201603	<cmath>
P0226R1	Mathematical Special Functions for C++17	29.9	_cpp_lib_math_special_functions ex.	201603	<cmath>
P0218R1	Adopt the File System TS for C++17	30.10	_has_include(<filesystem>) _cpp_lib_filesystem ex.	1 201603	<i>predefined</i> <filesystem>
P0219R1	Relative Paths for Filesystem	30.10	_cpp_lib_filesystem	201606	<filesystem>
P0392R0	Adapting string_view by filesystem paths	30.10	_cpp_lib_filesystem	201606	<filesystem>
P0317R1	Directory Entry Caching for Filesystem	30.10	_cpp_lib_filesystem	201703	<filesystem>
P0371R1	Temporarily discourage memory_order_consume	32.4	none		
P0152R1	constexpr atomic<T>::is_always_lock_free	32.6	_cpp_lib_atomic_is_always_lock_free ex.	201603	<atomic>
N4508	A proposal to add shared_mutex (untimed)	33.4	_cpp_lib_shared_mutex ex.	201505	<shared_mutex>
P0156R2	Variadic lock_guard (Rev. 5)	33.4	_cpp_lib_scoped_lock ex.	201703	<mutex>
P0004R1	Remove Deprecated iostreams aliases	D	none		

[3.5] C++14 features

[1] Ideally, these paragraphs should not be repeated.

[2] The following table itemizes all the changes that were made to the working draft for C++14 as specified in a WG21 technical document. (Changes that were made as specified in a core or library issue are not generally included.)

[3] The table is sorted by the section of the standard primarily affected. The “Doc. No.” column links to the paper itself on the committee web site. The “Macro Name” column links to the relevant portion of the “Detailed explanation and rationale” annex of this document. When the recommendation is to change the value of a macro previously recommended to be defined, the “Value” column links to the table entry for the previous recommendation.

[4] For library features, the “Header” column identifies the header that is expected to define the macro, although the macro may also be predefined. For language features, the macro must be predefined.

[5] Obviously, how far back to go with these tables will be an interesting question. From my perspective, it would be most useful to include all of them.

Significant changes to C++14

Doc. No.	Title	Primary Section	Macro Name	Value	Header
N3910	What can signal handlers do? (CWG 1441)	1.9-1.10	none		
N3927	Definition of Lock-Free	1.10	none		
N3472	Binary Literals in the C++ Core Language	2.14	__cpp_binary_literals	201304	<i>predefined</i>
N3781	Single-Quotation-Mark as a Digit Separator	2.14	none		
N3323	A Proposal to Tweak Certain C++ Contextual Conversions	4	none		
N3648	Wording Changes for Generalized Lambda-capture	5.1	__cpp_init_captures	201304	<i>predefined</i>
N3649	Generic (Polymorphic) Lambda Expressions	5.1	__cpp_generic_lambdas	201304	<i>predefined</i>
N3664	Clarifying Memory Allocation	5.3	none		
N3778	C++ Sized Deallocation	5.3, 18.6	__cpp_sized_deallocation	201309	<i>predefined</i>
N3624	Core Issue 1512: Pointer comparison vs qualification conversions	5.9, 5.10	none		
N3652	Relaxing constraints on constexpr functions / constexpr member functions and implicit const	5.19, 7.1	__cpp_constexpr	201304	<i>predefined</i>
N3638	Return type deduction for normal functions	7.1	__cpp_decltype_auto	201304	<i>predefined</i>
			__cpp_return_type_deduction	201304	<i>predefined</i>
N3760	[[deprecated]] attribute	7.6	_has_cpp_attribute(deprecated)	201309	<i>predefined</i>
N3653	Member initializers and aggregates	8.5	__cpp_aggregate_nsdmi	201304	<i>predefined</i>
N3667	Drafting for Core 1402	12.8	none		
N3651	Variable Templates	14, 14.7	__cpp_variable_templates	201304	<i>predefined</i>
N3669	Fixing constexpr member functions without const	various	none		
N3673	C++ Library Working Group Ready Issues Bristol 2013	various	none		
N3658	Compile-time integer sequences	20	__cpp_lib_integer_sequence	201304	<utility>
N3668	exchange() utility function	20	__cpp_lib_exchange_function	201304	<utility>
N3471	Constexpr Library Additions: utilities	20.2-20.4	none		
N3670	Wording for Addressing Tuples by Type	20.2-20.4	__cpp_lib_tuples_by_type	201304	<utility>
N3887	Consistent Metafunction Aliases	20.3-20.4	__cpp_lib_tuple_element_t	201402	<utility>
N3656	make_unique	20.7	__cpp_lib_make_unique	201304	<memory>
N3421	Making Operator Functors greater<>	20.8	__cpp_lib_transparent_operators	201210	<functional>
N3545	An Incremental Improvement to integral_constant	20.9	__cpp_lib_integral_constant_callable	201304	<type_traits>
N3655	TransformationTraits Redux	20.9	__cpp_lib_transformation_trait_aliases	201304	<type_traits>
N3789	Constexpr Library Additions: functional	20.10	none		
N3462	std::result_of and SFINAE	20.9 20.10	__cpp_lib_result_of_sfinae	201210	<functional> <type_traits>
LWG 2112	User-defined classes that cannot be derived from	20.10	__cpp_lib_is_final	201402	<type_traits>
LWG 2247	Type traits and std::nullptr_t	20.10	__cpp_lib_is_null_pointer	201309	<type_traits>
N3469	Constexpr Library Additions: chrono	20.11	none		
N3642	User-defined Literals for Standard Library Types	20.11	__cpp_lib_chrono_udls	201304	<chrono>
		21.7	__cpp_lib_string_udls	201304	<string>
N3470	Constexpr Library Additions: containers	23.3	none		
N3657	Adding heterogeneous comparison lookup to associative containers	23.4	__cpp_lib_generic_associative_lookup	201304	<map> <set>
N3644	Null Forward Iterators	24.2	__cpp_lib_null_iterators	201304	<iterator>
LWG 2285	make_reverse_iterator	24.5	__cpp_lib_make_reverse_iterator	201402	<iterator>
N3671	Making non-modifying sequence operations more robust	25.2	__cpp_lib_robust_nonmodifying_seq_ops	201304	<algorithm>
N3779	User-defined Literals for std::complex	26.4	__cpp_lib_complex_udls	201309	<complex>
N3924	Discouraging rand() in C++14	26.8	none		
N3654	Quoted Strings Library Proposal	27.7	__cpp_lib_quoted_string_io	201304	<iomanip>
LWG 2249	Remove gets from <cstdio>	27.9	none		
N3786	Prohibiting "out of thin air" results in C++14	29.3	none		
N3659	Shared locking in C++	30.4	_has_include(<shared_mutex>)	1	<i>predefined</i>
N3891	A proposal to rename shared_mutex to shared_timed_mutex	30.4	__cpp_lib_shared_timed_mutex	201402	<shared_mutex>
N3776	Wording for ~future	30.6	none		

[3.6] C++11 features

Significant features of C++11

Doc. No.	Title	Primary Section	Macro name	Value	Header
N2249	New Character Types in C++	2.13	<code>__cpp_unicode_characters</code>	200704	<i>predefined</i>
N2442	Raw and Unicode String Literals Unified Proposal	2.13	<code>__cpp_raw_strings</code>	200710	<i>predefined</i>
			<code>__cpp_unicode_literals</code>	200710	<i>predefined</i>
N2765	User-defined Literals	2.13, 13.5	<code>__cpp_user_defined_literals</code>	200809	<i>predefined</i>
N2660	Dynamic Initialization and Destruction with Concurrency	3.6	<code>__cpp_threadsafe_static_init</code>	200806	<i>predefined</i>
N2927	New wording for C++0x lambdas	5.1	<code>__cpp_lambdas</code>	200907	<i>predefined</i>
N2235	Generalized Constant Expressions	5.19, 7.1	<code>__cpp_constexpr</code>	200704	<i>predefined</i>
N2930	Range-Based For Loop Wording (Without Concepts)	6.5	<code>__cpp_range_based_for</code>	200907	<i>predefined</i>
N1720	Proposal to Add Static Assertions to the Core Language	7	<code>__cpp_static_assert</code>	200410	<i>predefined</i>
N2343	Decltype	7.1	<code>__cpp_decltype</code>	200707	<i>predefined</i>
N2761	Towards support for attributes in C++	7.6	<code>__cpp_attributes</code>	200809	<i>predefined</i>
			<code>__has_cpp_attribute(noreturn)</code>	200809	<i>predefined</i>
			<code>__has_cpp_attribute(carries_dependency)</code>	200809	<i>predefined</i>
N2118	A Proposal to Add an Rvalue Reference to the C++ Language	8.3	<code>__cpp_rvalue_references</code>	200610	<i>predefined</i>
N2242	Proposed Wording for Variadic Templates	8.3, 14	<code>__cpp_variadic_templates</code>	200704	<i>predefined</i>
N2672	Initializer List proposed wording	8.5	<code>__cpp_initializer_lists</code>	200806	<i>predefined</i>
N1986	Delegating Constructors	12.6	<code>__cpp_delegating_constructors</code>	200604	<i>predefined</i>
N2756	Non-static data member initializers	12.6	<code>__cpp_nsdmi</code>	200809	<i>predefined</i>
N2540	Inheriting Constructors	12.9	<code>__cpp_inheriting_constructors</code>	200802	<i>predefined</i>
N2439	Extending move semantics to *this	13.3	<code>__cpp_ref_qualifiers</code>	200710	<i>predefined</i>
N2258	Template Aliases	14.5	<code>__cpp_alias_templates</code>	200704	<i>predefined</i>

[3.7] C++98 features

[1] With very few exceptions, the features of C++98 have all been implemented in virtually every C++ compiler. But in many compilers, some of them can be enabled/disabled. It is recommended that the macros in the following table should be used to indicate whether one of these features is enabled in the current compilation.

Feature	Primary section	Macro name	Value	Header
Run-time type identification	5.2	<code>__cpp_rtti</code>	199711	<i>predefined</i>
Exception handling	15	<code>__cpp_exceptions</code>	199711	<i>predefined</i>

[3.8] Features published and later removed

[1] SG10 considered the stability of these recommendations to be very important, hence these tables. I believe that, going forward with having the macros in the WD, having information about features once included but later removed will still be necessary or desirable. But it may make more sense to have this information in SD-6 instead of the WD.

[3.8.1] Features removed from C++17

Features in the CD of C++17, subsequently removed

Doc. No.	Title	Primary Section	Macro Name	Value	Header
P0077R2	is_callable, the missing INVOKE related trait	20.15	<code>__cpp_lib_is_callable</code>	201603	<type_traits>
P0181R1	Ordered By Default	20.14, 23.4, 23.6	<code>__cpp_lib_default_order</code>	201606	<type_traits>
P0156R0	Variadic lock_guard	30.4	<code>__cpp_lib_lock_guard_variadic</code>	201510	<mutex>

[3.8.2] Features removed from C++14

Features in the first CD of C++14, subsequently removed to a Technical Specification

Doc. No.	Title	Primary Section	Macro Name	Value	Header
N3639	Runtime-sized arrays with automatic storage duration	8.3	<code>__cpp_runtime_arrays</code>	201304	<i>predefined</i>
N3672	A proposal to add a utility class to represent optional objects	20.5	<code>__has_include(<optional>)</code>	1	<i>predefined</i>
N3662	C++ Dynamic Arrays	23.2, 23.3	<code>__has_include(<dynarray>)</code>	1	<i>predefined</i>

[1] The intention is that an implementation which provides runtime-sized arrays as specified in the first CD should define `__cpp_runtime_arrays` as 201304. The expectation is that a later document specifying runtime-sized arrays will specify a different value for `__cpp_runtime_arrays`.

[A] Annex: Recommendations from Technical Specifications (informative)

[2] These annexes to SD-6 would probably make no sense in the WD.

[3] In this annex, feature-test macros from all WG21 Technical Specifications are collected, for convenience.

[A.9] C++ Extensions for Library Fundamentals

[1] The recommended macro name is "`__cpp_lib_experimental_`" followed by the string in the "Macro Name Suffix" column.

Doc. No.	Title	Primary Section	Macro Name Suffix	Value	Header
N3915	apply() call a function with arguments from a tuple	3.2.2	apply	201402	<experimental/tuple>
N3932	Variable Templates For Type Traits	3.3.1	type_trait_variable_templates	201402	<experimental/type_traits>
N3866	Invocation type traits	3.3.2	invocation_type	201406	<experimental/type_traits>
N3916	Type-erased allocator for std::function	4.2	function_erased_allocator	201406	<experimental/functional>
N3905	Extending std::search to use Additional Searching Algorithms	4.3	boyer_moore_searching	201411	<experimental/functional>
N3672, N3793	A utility class to represent optional objects	5	optional	201411	<experimental/optional>
N3804	Any Library Proposal	6	any	201411	<experimental/any>
N3921	string_view: a non-owning reference to a string	7	string_view	201411	<experimental/string_view>
N3920	Extending shared_ptr to Support Arrays	8.2	shared_ptr_arrays	201406	<experimental/memory>
N3916	Polymorphic Memory Resources	8.4	memory_resources	201402	<experimental/memory_resource>
N3916	Type-erased allocator for std::promise	9.2	promise_erased_allocator	201406	<experimental/future>
N3916	Type-erased allocator for std::packaged_task	9.3	packaged_task_erased_allocator	201406	<experimental/future>
N3925	A sample Proposal	10.3	sample	201402	<experimental/algorithm>

[A.10] C++ Extensions for Parallelism

Name	Value	Header
<code>__cpp_lib_experimental_parallel_algorithm</code>	201505	<experimental/algorithm> <experimental/exception_list> <experimental/execution_policy> <experimental/numeric>

[A.11] File System Technical Specification

Name	Value	Header
<code>__cpp_lib_experimental_filesystem</code>	201406	<experimental/filesystem>

[A.12] C++ Extensions for Transactional Memory

Name	Value	Header
<code>__cpp_transactional_memory</code>	201505	<i>predeclared</i>
<code>__has_cpp_attribute(optimize_for_synchronized)</code>	201505	<i>predeclared</i>

[1] The Technical Specification for transactional memory specifies a new attribute, but does not specify a value for a *has-attribute-expression* for that attribute.

[B] Annex: Model wording for a Technical Specification

[1] It is recommended that Technical Specifications include the wording below. *yyyymm* indicates the year and month of the date of the TS.

[2] 1.x Feature testing [intro.features]

[3] An implementation that provides support for this Technical Specification shall define the feature test macro(s) in Table X.

Table X — Feature Test Macro(s)

Name	Value	Header
<code>__cpp_name</code>	<i>yyyymm</i>	<i>“predefined” OR “<name>”</i>

[C] Revision history

Date	Document	Description
	P0941R0	Initial draft